# Lab Assignment #10

Math 437 - Modern Data Analysis - Jason Soto

### Instructions

The purpose of this lab is to introduce several different classification strategies and variations on classification accuracy. In this lab we will work with, k-nearest neighbors, Naive Bayes and linear/quadratic discriminant analysis.

```
here::i_am("Labs/Lab10.qmd")

# These are loaded in Lab 4.7
library(ISLR2)
library(MASS) # LDA/QDA
library(e1071) # Naive Bayes
library(class) # knn

# Might need these packages if you use tidyverse/tidymodels
library(tidyverse)
library(tidymodels)
library(discrim)
library(klaR)
library(kknn)

# compas data
compas <- readr::read_csv(here::here("Data/compas.csv"))
# factoring is done, no need to refactor
compas$recid <- factor(compas$recid,
                       levels = c("yes", "no"))
compas_train <- compas[which(compas$screening_year == 2013), ]
compas_test <- compas[which(compas$screening_year == 2014), ]
```

This lab assignment is worth a total of **17.5 points**.

For Problems 2-3, you may fit and predict using either the regular functions or the `tidymodels` workflows. Please do not mix and match. You should use functions from the `yardstick` package to do all model assessment.

## Problem 1: Understanding the Classification Algorithms

### Part a (Code: 4 pts)

Run the code in ISLR Labs 4.7.3-4.7.6. Put each chunk from the textbook in its own chunk. As usual, read the text to figure out what the chunks do, and don't `attach` anything (use the `with` function that we've learned about instead, e.g., `with(Smarket, plot(Volume))` instead of attaching Smarket).

```
train <- (Smarket$Year < 2005)
Smarket.2005 <- Smarket[!train, ]
Direction.2005 <- Smarket$Direction[!train]
```

### Section 4.7.3

(COMMENTED OUT PLOT, QUARTO WONT RENDER, SAYS figure margins too large)

```
library(MASS)
lda.fit <- lda(Direction ~ Lag1 + Lag2,
               data = Smarket,
               subset = train)
lda.fit
```

```
Call:
lda(Direction ~ Lag1 + Lag2, data = Smarket, subset = train)

Prior probabilities of groups:
    Down       Up
0.491984 0.508016

Group means:
            Lag1        Lag2
Down  0.04279022  0.03389409
Up   -0.03954635 -0.03132544

Coefficients of linear discriminants:
```

```
            LD1
Lag1 -0.6420190
Lag2 -0.5135293
```

```
#plot(lda.fit)
```

```
lda.pred <- predict(lda.fit, Smarket.2005)
names(lda.pred)
```

```
[1] "class"     "posterior" "x"
```

```
lda.class <- lda.pred$class
table(lda.class, Direction.2005)
```

```
         Direction.2005
lda.class Down  Up
     Down   35  35
     Up     76 106
```

```
mean(lda.class == Direction.2005)
```

```
[1] 0.5595238
```

```
sum(lda.pred$posterior[, 1] >= .5)
```

```
[1] 70
```

```
sum(lda.pred$posterior[, 1] < .5)
```

```
[1] 182
```

```
lda.pred$posterior[1:20, 1]
```

```
      999      1000      1001      1002      1003      1004      1005      1006
0.4901792 0.4792185 0.4668185 0.4740011 0.4927877 0.4938562 0.4951016 0.4872861
     1007      1008      1009      1010      1011      1012      1013      1014
0.4907013 0.4844026 0.4906963 0.5119988 0.4895152 0.4706761 0.4744593 0.4799583
     1015      1016      1017      1018
0.4935775 0.5030894 0.4978806 0.4886331
```

```
lda.class[1:20]
```

```
 [1] Up   Up   Up   Up   Up   Up   Up   Up   Up   Up   Up   Down Up   Up   Up
[16] Up   Up   Down Up   Up
Levels: Down Up
```

```
sum(lda.pred$posterior[, 1] > .9)
```

```
[1] 0
```

**Section 4.7.4**

```
qda.fit <- qda(Direction ~ Lag1 + Lag2,
               data = Smarket,
               subset = train)

qda.fit
```

```
Call:
qda(Direction ~ Lag1 + Lag2, data = Smarket, subset = train)

Prior probabilities of groups:
    Down       Up
0.491984 0.508016

Group means:
            Lag1        Lag2
Down  0.04279022  0.03389409
Up   -0.03954635 -0.03132544
```

```
qda.class <- predict(qda.fit, Smarket.2005)$class
table(qda.class, Direction.2005)
```

```
         Direction.2005
qda.class Down  Up
     Down   30  20
     Up     81 121
```

```
mean(qda.class == Direction.2005)
```

```
[1] 0.5992063
```

### Section 4.7.5

```
library(e1071)

nb.fit <- naiveBayes(Direction ~ Lag1 + Lag2,
                     data = Smarket,
                     subset = train)
nb.fit
```

```
Naive Bayes Classifier for Discrete Predictors

Call:
naiveBayes.default(x = X, y = Y, laplace = laplace)

A-priori probabilities:
Y
    Down       Up
0.491984 0.508016

Conditional probabilities:
      Lag1
Y             [,1]     [,2]
  Down  0.04279022 1.227446
  Up   -0.03954635 1.231668

      Lag2
Y             [,1]     [,2]
  Down  0.03389409 1.239191
  Up   -0.03132544 1.220765
```

```
mean(Smarket$Lag1[train][Smarket$Direction[train] == "Down"])
```

```
[1] 0.04279022
```

```
sd(Smarket$Lag1[train][Smarket$Direction[train] == "Down"])
```

```
[1] 1.227446
```

```
nb.class <- predict(nb.fit, Smarket.2005)
table(nb.class, Direction.2005)
```

```
        Direction.2005
nb.class Down  Up
    Down   28  20
    Up     83 121
```

```
mean(nb.class == Direction.2005)
```

```
[1] 0.5912698
```

```
nb.preds <- predict(nb.fit, Smarket.2005, type = "raw")

nb.preds[1:5,]
```

```
          Down        Up
[1,] 0.4873164 0.5126836
[2,] 0.4762492 0.5237508
[3,] 0.4653377 0.5346623
[4,] 0.4748652 0.5251348
[5,] 0.4901890 0.5098110
```

### Section 4.7.6

```
library(class)

train.X <- cbind(Smarket$Lag1, Smarket$Lag2)[train, ]

test.X <- cbind(Smarket$Lag1, Smarket$Lag2)[!train, ]

train.Direction <- Smarket$Direction[train]
```

```
set.seed(1)
knn.pred <- knn(train.X, test.X, train.Direction, k = 1)

table(knn.pred, Direction.2005)
```

```
        Direction.2005
knn.pred Down Up
    Down   43 58
    Up     68 83
```

```
(83 + 43) / 252
```

```
[1] 0.5
```

```
knn.pred <- knn(train.X, test.X, train.Direction, k = 3)

table(knn.pred, Direction.2005)
```

```
        Direction.2005
knn.pred Down Up
    Down   48 54
    Up     63 87
```

```
mean(knn.pred == Direction.2005)
```

```
[1] 0.5357143
```

```
dim(Caravan)
```

```
[1] 5822   86
```

```
with(Caravan, summary(Purchase))
```

```
  No  Yes
5474  348
```

```
348 / 5822
```

```
[1] 0.05977327
```

```
standardized.X <- scale(Caravan[, -86])

var(Caravan[, 1])
```

```
[1] 165.0378
```

```
var(Caravan[, 2])
```

```
[1] 0.1647078
```

```
var(standardized.X[, 1])
```

```
[1] 1
```

```
var(standardized.X[, 2])
```

```
[1] 1
```

```
test <- 1:1000

train.X <- standardized.X[-test, ]
test.X <- standardized.X[test, ]

train.Y <- Caravan$Purchase[-test]
test.Y <- Caravan$Purchase[test]

set.seed(1)

knn.pred <- knn(train.X, test.X, train.Y, k = 1)

mean(test.Y != knn.pred)
```

```
[1] 0.118
```

```r
mean(test.Y != "No")
```

```
[1] 0.059
```

```r
table(knn.pred, test.Y)
```

```
        test.Y
knn.pred  No Yes
     No  873  50
     Yes  68   9
```

```r
9 / (68 + 9)
```

```
[1] 0.1168831
```

```r
knn.pred <- knn(train.X, test.X, train.Y, k = 3)
table(knn.pred, test.Y)
```

```
        test.Y
knn.pred  No Yes
     No  920  54
     Yes  21   5
```

```r
5 / 26
```

```
[1] 0.1923077
```

```r
knn.pred <- knn(train.X, test.X, train.Y, k = 5)
table(knn.pred, test.Y)
```

```
        test.Y
knn.pred  No Yes
     No  930  55
     Yes  11   4
```

```
4 / 15
```

```
[1] 0.2666667
```

```
glm.fits <- glm(Purchase ~ .,
                data = Caravan,
                family = binomial,
                subset = -test)
```

```
Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
glm.probs <- predict(glm.fits, Caravan[test, ], type = "response")

glm.pred <- rep("No", 1000)
glm.pred[glm.probs > .5] <- "Yes"
table(glm.pred, test.Y)
```

```
        test.Y
glm.pred  No Yes
     No  934  59
     Yes   7   0
```

```
glm.pred <- rep("No", 1000)
glm.pred[glm.probs > .25] <- "Yes"
table(glm.pred, test.Y)
```

```
        test.Y
glm.pred  No Yes
     No  919  48
     Yes  22  11
```

```
11 / (22 + 11)
```

```
[1] 0.3333333
```

### Problem 2: Recidivism Prediction

Just like in Lab 5, we will use additional classification models to predict whether an arrested person will reoffend within two years.

### Part a (Code: 3.5 pts; Explanation: 1 pt)

Choose an accuracy metric to optimize (e.g., AUC, mean log-loss). Using 5-fold cross-validation on the training set, determine and justify an "optimal" value for the number of nearest neighbors in a k-nearest neighbors model. Support your decision by creating a plot showing the average accuracy as a function of `k`.

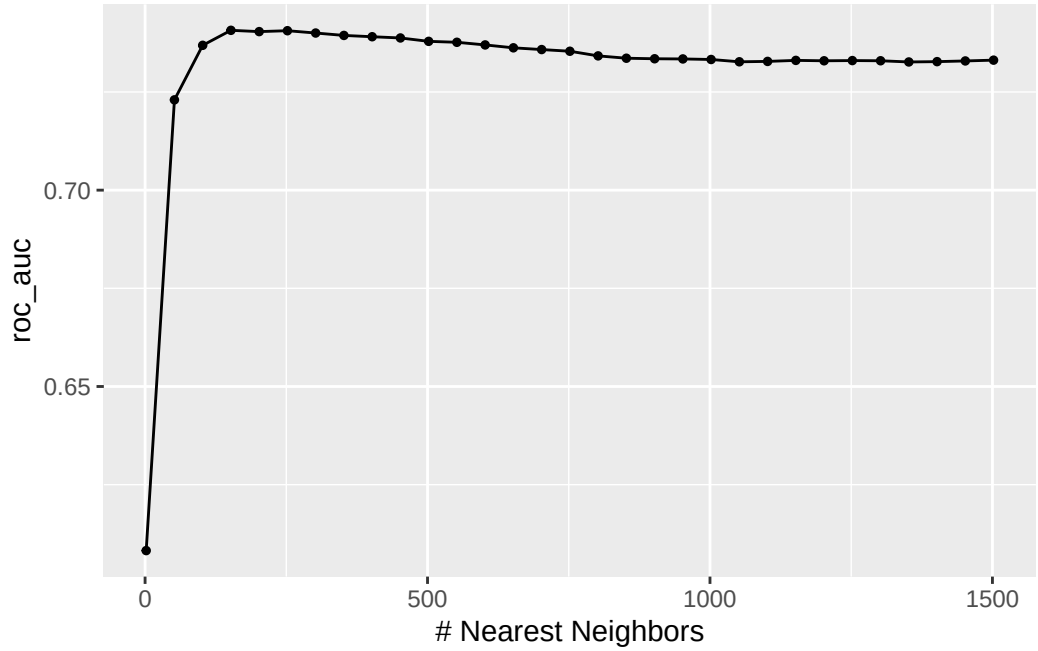
If you are using "regular R", Euclidean distance is used by default, and you will need two `for` loops (an outer loop over the folds and an inner loop over the possible values of `k`). To avoid confusion with `k` in k-fold vs `k` in k-nn, use `v` as the fold index.

If you are using `tidymodels`, you should create a `model` specifying the algorithm and the parameter to tune, then create a grid of possible parameter values and tune over it. You should set `dist_power = 2` and not worry about tuning it.

#### Solution

```
# Setting up model for knn classification
knn_model <- nearest_neighbor(
  mode = "classification",
  neighbors = tune(),
  dist_power = 2
)

# Standard recipe with normalization
knn_recipe <- recipe(
  recid ~ age_at_screening + sex + priors,
  data = compas_train
) |>
  step_dummy(all_nominal_predictors()) |>
  step_normalize(all_numeric_predictors())

# Basic Workflow
knn_wflow <- workflow() |>
  add_model(knn_model) |>
  add_recipe(knn_recipe)

set.seed(676767)

# Defines/plans 5 Fold Cross Validation, strata is response variable
knn_kfold <- vfold_cv(compas_train,
                      v = 5,
                     strata = recid)
```

```r
# Parameter grid, sets up which k's to try
# Take one value, 5-fold CV, compute metric, then repeat
knn_grid <- expand.grid(
  neighbors = seq(2, 1502, by = 50)     # Keep testing until you get the roc auc pattern we a
)

# Performs 5 Fold and computes metric, averages the results across folds
knn_tune <- tune_grid(
  knn_wflow,     # workflow to tune
  resamples = knn_kfold,     # tibble with instructions to k-fold cv
  grid = knn_grid,     # grid of hyperparameters values to search over
  metrics = metric_set(roc_auc)
)

# Plot
autoplot(knn_tune)
```




```r
# Best k
best_k <- select_best(knn_tune, metric = "roc_auc")
best_k
```

```
# A tibble: 1 x 2
```

```
  neighbors .config
      <dbl> <chr>
1       152 pre0_mod04_post0
```

I used ROC AUC for the accuracy metric and with 5-fold cross-validation. Based on the plot and code, its peak is around 152 nearest neighbors, then it starts to level off. That is why I chose K to be 152, though in the code I left it as `best_k <- select_best(knn_tune, metric = "roc_auc")` for ease of use and stuff.

### Part b (Code: 1.5 pts)

Fit a k-nearest neighbors model on the full `compas_train` training set using the value of `k` selected in Part a.

Then, using this model, predict whether each arrested person in the `compas_test` dataset will be arrested within two years of release (`recid == yes`). Also, create a `compas_knn_predictions` data frame or tibble containing both the predicted and actual classes and the predicted probabilities of "yes" and "no".

Remember to re-scale the predictor variables in your test set based on the full training set!

### Solution

```
knn_model_final <- nearest_neighbor(
  mode = "classification",
  neighbors = best_k$neighbors,     # hard coded instead of select best is done here
  dist_power = 2
)

knn_wflow_final <- workflow() |>
  add_model(knn_model_final) |>
  add_recipe(knn_recipe)

knn_fit_tuned <- fit(knn_wflow_final,
                     data = compas_train)

compas_knn_predictions <- knn_fit_tuned |>
  augment(new_data = compas_test)

compas_knn_predictions
```

```
# A tibble: 1,016 x 10
   .pred_class .pred_yes .pred_no screening_year age_at_screening priors sex
   <fct>           <dbl>    <dbl>          <dbl>             <dbl>  <dbl> <chr>
 1 no             0.0682    0.932           2014                37      0 Female
 2 no             0.122     0.878           2014                35      0 Male
 3 no             0.106     0.894           2014                31      0 Male
 4 yes            0.579     0.421           2014                23      9 Male
 5 no             0.446     0.554           2014                33     13 Male
 6 no             0.0210    0.979           2014                61      1 Female
 7 no             0.106     0.894           2014                31      0 Male
 8 no             0.113     0.887           2014                25      1 Male
 9 yes            0.579     0.421           2014                23      9 Male
10 no             0.345     0.655           2014                18      1 Male
# i 1,006 more rows
# i 3 more variables: race <chr>, recid <fct>, compas_recid <chr>
```

```r
conf_mat(
  compas_knn_predictions,
  truth = recid,
  estimate = .pred_class
)
```

```
          Truth
Prediction yes  no
       yes  28   5
       no  284 699
```

**Part c (Code: 1.5 pts)**

Fit a Naive Bayes model on the `compas_train` training set predicting `recid` from `age_at_screening`, `priors`, and `sex`. Remember that you will have to convert `sex` to a factor variable (or use `step_dummy`) if you want to use tidymodels!

Then, using this model, predict whether each arrested person in the `compas_test` dataset will be arrested within two years of release (`recid == yes`). Also, create a `compas_nb_predictions` data frame or tibble containing both the predicted and actual classes and the predicted probabilities of "yes" and "no".

**Solution**

```r
nb_model <- naive_Bayes(
  mode = "classification",
  engine = "klaR"
  # both are defaults
)

nb_recipe <- recipe(
  recid ~ age_at_screening + priors + sex,
  data = compas_train
)
# Requires factor variables of categorical - NO DUMMY VARIABLES!

nb_wflow <- workflow() |>
  add_model(nb_model) |>
  add_recipe(nb_recipe)

nb_fit <- nb_wflow |>
  fit(data = compas_train)

compas_nb_predictions <- nb_fit |>
  augment(new_data = compas_test)

compas_nb_predictions
```

```
# A tibble: 1,016 x 10
   .pred_class .pred_yes .pred_no screening_year age_at_screening priors sex
   <fct>           <dbl>    <dbl>          <dbl>             <dbl>  <dbl> <chr>
 1 no             0.0247    0.975           2014                37      0 Female
 2 no             0.0596    0.940           2014                35      0 Male
 3 no             0.0721    0.928           2014                31      0 Male
 4 yes            0.503     0.497           2014                23      9 Male
 5 no             0.397     0.603           2014                33     13 Male
 6 no             0.0107    0.989           2014                61      1 Female
 7 no             0.0721    0.928           2014                31      0 Male
 8 no             0.150     0.850           2014                25      1 Male
 9 yes            0.503     0.497           2014                23      9 Male
10 no             0.187     0.813           2014                18      1 Male
# i 1,006 more rows
# i 3 more variables: race <chr>, recid <fct>, compas_recid <chr>
```

```r
conf_mat(
  compas_nb_predictions,
  truth = recid,
  estimate = .pred_class
)
```

```
          Truth
Prediction yes  no
       yes   8   2
       no  304 702
```

### Part d (Code: 1.5 pts)

Fit a linear discriminant analysis model on the training set predicting `recid` from `age_at_screening`, `priors`, and `sex`. Then, using this model, predict whether each arrested person in the `compas_test` dataset will be arrested within two years of release (`recid == yes`). Also, create a `compas_lda_predictions` data frame or tibble containing both the predicted and actual classes and the predicted probabilities of "yes" and "no".

### Solution

```r
lda_model <- discrim_linear(
  mode = "classification",      # default
  engine = "MASS"     # default
)

lda_wflow <- workflow() |>
  add_model(lda_model) |>
  add_recipe(nb_recipe)

lda_fit <- lda_wflow |>
  fit(data = compas_train)

compas_lda_predictions <- lda_fit |>
  augment(new_data = compas_test)

compas_lda_predictions
```

```
# A tibble: 1,016 x 10
   .pred_class .pred_yes .pred_no screening_year age_at_screening priors sex
```

```
   <fct>           <dbl>    <dbl>          <dbl>            <dbl>  <dbl> <chr>
 1 no             0.0601    0.940           2014               37      0 Female
 2 no             0.104     0.896           2014               35      0 Male
 3 no             0.122     0.878           2014               31      0 Male
 4 no             0.491     0.509           2014               23      9 Male
 5 yes            0.554     0.446           2014               33     13 Male
 6 no             0.0253    0.975           2014               61      1 Female
 7 no             0.122     0.878           2014               31      0 Male
 8 no             0.178     0.822           2014               25      1 Male
 9 no             0.491     0.509           2014               23      9 Male
10 no             0.228     0.772           2014               18      1 Male
# i 1,006 more rows
# i 3 more variables: race <chr>, recid <fct>, compas_recid <chr>
```

```r
conf_mat(
  compas_lda_predictions,
  truth = recid,
  estimate = .pred_class
)
```

```
          Truth
Prediction yes  no
       yes  29  18
       no  283 686
```

### Part e (Code: 1.5 pts)

Fit a quadratic discriminant analysis model on the training set predicting `recid` from `age_at_screening`, `priors`, and `sex`. Remember that you will have to convert `sex` to a factor variable (or use `step_dummy`) if you use tidymodels!

Then, using this model, predict whether each arrested person in the `compas_test` dataset will be arrested within two years of release (`recid == yes`). Also, create a `compas_qda_predictions` data frame or tibble containing both the predicted and actual classes and the predicted probabilities of "yes" and "no".

### Solution

```r
qda_model <- discrim_quad(
  mode = "classification",      # default
  engine = "MASS"     # default
)

qda_wflow <- workflow() |>
  add_model(qda_model) |>
  add_recipe(nb_recipe)

qda_fit <- qda_wflow |>
  fit(data = compas_train)

compas_qda_predictions <- qda_fit |>
  augment(new_data = compas_test)

compas_qda_predictions
```

```
# A tibble: 1,016 x 10
   .pred_class .pred_yes .pred_no screening_year age_at_screening priors sex
   <fct>           <dbl>    <dbl>          <dbl>             <dbl>  <dbl> <chr>
 1 no            0.0218     0.978           2014                37      0 Female
 2 no            0.143      0.857           2014                35      0 Male
 3 no            0.173      0.827           2014                31      0 Male
 4 yes           0.622      0.378           2014                23      9 Male
 5 yes           0.870      0.130           2014                33     13 Male
 6 no            0.00169    0.998           2014                61      1 Female
 7 no            0.173      0.827           2014                31      0 Male
 8 no            0.218      0.782           2014                25      1 Male
 9 yes           0.622      0.378           2014                23      9 Male
10 no            0.238      0.762           2014                18      1 Male
# i 1,006 more rows
# i 3 more variables: race <chr>, recid <fct>, compas_recid <chr>
```

```r
conf_mat(
  compas_qda_predictions,
  truth = recid,
  estimate = .pred_class
)
```

```
          Truth
Prediction yes  no
```

```
       yes  51  25
       no  261 679
```

## Problem 3: Evaluating Probabilistic Predictions

### Part a (Code: 0.5 pts; Explanation: 1 pt)

Using the `brier_class` function, obtain the Brier score for each model.

Which of the four models performs best by this measure? Which performs worst? Explain.

### Solution

```
brier_class(compas_knn_predictions, truth = recid, .pred_yes)
```

```
# A tibble: 1 x 3
  .metric     .estimator .estimate
  <chr>       <chr>          <dbl>
1 brier_class binary         0.204
```

```
brier_class(compas_nb_predictions, truth = recid, .pred_yes)
```

```
# A tibble: 1 x 3
  .metric     .estimator .estimate
  <chr>       <chr>          <dbl>
1 brier_class binary         0.209
```

```
brier_class(compas_lda_predictions, truth = recid, .pred_yes)
```

```
# A tibble: 1 x 3
  .metric     .estimator .estimate
  <chr>       <chr>          <dbl>
1 brier_class binary         0.204
```

```
brier_class(compas_qda_predictions, truth = recid, .pred_yes)
```

```
# A tibble: 1 x 3
  .metric     .estimator .estimate
  <chr>       <chr>          <dbl>
1 brier_class binary         0.210
```

For Brier Class, the lowest brier score is the best while the highest is the worst. So, our best model is LDA with a brier score of about .2038. Our worst model is QDA with a brier score of about .2097.

### Part b (Code: 0.5 pts; Explanation: 1 pt)

Using the `mn_log_loss` function, obtain the cross-entropy/log loss for each model.

Which of the four models performs best by this measure? Which performs worst? Explain.

### Solution

```
mn_log_loss(compas_knn_predictions, truth = recid, .pred_yes, event_level = "first")
```

```
# A tibble: 1 x 3
  .metric     .estimator .estimate
  <chr>       <chr>          <dbl>
1 mn_log_loss binary         0.625
```

```
mn_log_loss(compas_nb_predictions, truth = recid, .pred_yes, event_level = "first")
```

```
# A tibble: 1 x 3
  .metric     .estimator .estimate
  <chr>       <chr>          <dbl>
1 mn_log_loss binary         0.638
```

```
mn_log_loss(compas_lda_predictions, truth = recid, .pred_yes, event_level = "first")
```

```
# A tibble: 1 x 3
  .metric     .estimator .estimate
  <chr>       <chr>          <dbl>
1 mn_log_loss binary         0.603
```

```
mn_log_loss(compas_qda_predictions, truth = recid, .pred_yes, event_level = "first")
```

```
# A tibble: 1 x 3
  .metric     .estimator .estimate
  <chr>       <chr>          <dbl>
1 mn_log_loss binary         0.678
```

For mn_log_loss, the lowest .estimate value is the best while the highest is the worst. So, our best model is LDA with an estimate value of about .6033. Our worst model is QDA with an estimate value of about .6782.